

```
const char *dev_name, //contains the owner of the interrupt
void *dev_id //to identify which device is interrupting
);

void free_irq(unsigned int irq, void *dev_id);
```

Here, *request_irq* returns either 0 to indicate success, or a negative error code. Every time a timer interrupt occurs, the value of an internal kernel counter is incremented. The counter is initialised to 0 at system boot, so it represents the number of clock ticks since the last boot. The counter is a 64-bit variable (even on 32-bit architectures) and is called *jiffies_64*.

Jiffies

The global variable, *jiffies*, holds the number of ticks that have occurred since the system booted. On boot, the kernel initialises the variable to zero, and it is incremented by one during each timer interrupt. Thus, because there are Hz timer interrupts in a second, there are Hz *jiffies* in a second. The system uptime is therefore *jiffies*/Hz seconds. What actually happens is slightly more complicated. The kernel initialises *jiffies* to a special initial value, causing the variable to overflow more often, catching bugs. When the actual value of *jiffies* is sought, this ‘offset’ is first subtracted. The *jiffies* variable is declared in `<linux/jiffies.h>` as *extern unsigned long volatile jiffies*; one should generally use `<linux/sched.h>`, which automatically pulls in `<jiffies.h>` to use the counter and the utility functions.

Calculating system date

The current time of day (the wall time) is defined in *kernel/time/timekeeping.c*. The structures responsible for fetching the system data are as follows:

```
struct timespec {
    __kernel_time_t tv_sec; /* seconds */
    long tv_nsec; /* nanoseconds */
};
/* __kernel_time_t is long type mentioned in posix_types.h*/

struct timeval {
    __kernel_time_t tv_sec; /* seconds */
    __kernel_suseconds_t tv_usec; /* microseconds */
};

struct timezone {
    int tz_minuteswest; /* minutes west of Greenwich */
    int tz_dsttime; /* type of dst correction */
};
```

The *timeval*, *timespec* and *timezone* data structures are defined in `<linux/time.h>`. The *xtime.tv_sec* value stores the number of seconds that have elapsed since January 1, 1970 (UTC). This date is called the *epoch*. In most UNIX

systems, the current wall time is set relative to this epoch. The *xtime.tv_nsec* value stores the number of nanoseconds that have elapsed in the last second. The date is fetched into the structure by calling a routine called *getnstimeofday*, defined in *kernel/timekeeping.c* as follows:

```
/* Returns the time of day in a timeval*/
void do_gettimeofday(struct timeval *tv) {
    struct timespec now;
    getnstimeofday(&now); /*Returns the time of day in a timespec*/
    tv->tv_sec = now.tv_sec;
    tv->tv_usec = now.tv_nsec/1000;
}
```

Kernel code (especially a driver) often needs a way to delay execution for some time without using timers. This is usually to allow the hardware time to complete a given task. The time is typically quite short. For example, the specifications for a network card might list the time to change Ethernet modes as two microseconds. After setting the desired speed, the driver should wait at least for two microseconds before continuing.

Long and short delays

Long delays: Occasionally, a driver needs to delay execution for relatively long periods—more than one clock tick. Some solutions hog the processor while retarding real work, while other solutions do not do so, but offer no guarantee that your code will resume in exactly the required time. There are certain approaches regarding a long delay, which are listed below:

- The brain-dead approach: The simplest and easiest implementation is *busy waiting* or *busy looping*. It is applied as follows:

```
unsigned long j = jiffies + jit_delay * Hz;
while (jiffies < j)
    /* void */;
```

However, this technique precludes the CPU from performing any other tasks, since the loop continues till the *jiffies* reach *jit_delay*.

- The scheduling approach: This explicitly releases the CPU by using the *schedule* function declared in `<linux/sched.h>`

```
while (time_before(jiffies, j1)) {
    schedule();
}
```

- The timeout approach: The best way to implement a long delay is to use the kernel’s intelligence. If, say, a driver uses a wait queue for some event, and simultaneously wants the event to be completed within a particular time span, then one should use *wait_event_timeout*, declared in `<linux/wait.h>` as follows: